NEXUS OS: The Ultimate Open Source Digital Creation Ecosystem

Complete End-to-End Product Requirements Document

📋 Table of Contents

1. Executive Summary
2. Product Vision & Philosophy
3. System Architecture
4. Core Modules (A-Z)
5. Technical Stack
6. Database Schema
7. API Specifications
8. Frontend Architecture
9. Security Implementation
10. Open Source Strategy
11. Development Roadmap
12. Deployment Infrastructure
13. Testing Strategy
14. Monitoring & Observability
15. Monetization Model
16. Risk Management
17. Success Metrics
18. Community Guidelines
19. Documentation Requirements
20. Launch Checklist

---

1. Executive Summary

NEXUS OS is a revolutionary open-source platform that combines project management, AI orchestration, marketplace functionality, Web3 capabilities, and collaborative tools into a unified ecosystem. It's designed to be the operating system for digital creators, freelancers, entrepreneurs, and small businesses—providing everything needed to ideate, build, launch, and scale digital ventures from a single interface.

Key Differentiators

- **100% Open Source** with AGPL license
- **Tokenized Contribution Economy** rewarding community involvement
- **AI-First Architecture** with human-in-the-loop validation
- **Decentralized Storage** via IPFS/Filecoin
- **Quantum-Safe Encryption** for enterprise-grade security
- **Multi-Platform Support** (Web, Desktop, Mobile, CLI)
- **Built-in Marketplace** for buying/selling digital assets
- **DAO Governance** for community-driven development

---

2. Product Vision & Philosophy

Mission Statement

"Democratize digital creation by providing a free, open-source, AI-powered ecosystem that empowers anyone to build, manage, and scale their digital empire without technical barriers."

Core Principles

1. **Open by Default:** Every line of code is public, auditable, and forkable
2. **AI-Augmented, Not AI-Replaced:** AI assists but humans remain in control
3. **Community-Owned:** Users and contributors share in the platform's success
4. **Privacy-First:** User data is encrypted and owned by the user
5. **Interoperable:** Works with existing tools and standards
6. **Sustainable:** Built-in monetization ensures long-term viability

Target Audience

Persona	Use Case	Key Features
Indie Hackers	Build and launch SaaS products	Domain validator, task management, marketplace
Freelancers	Manage clients and projects	Jira clone, invoicing, portfolio builder
Small Business	Digital presence management	Website builder, SEO tools, social media bots
Developers	Build open source tools	Code playground, API access, CLI tools
Creators	Content creation & monetization	Autoblogger, community forum, NFTs
Agencies	Manage multiple clients	White-label, team management, analytics

---

3. System Architecture

High-Level Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                              CLIENT LAYER                            │
├──────────────┬──────────────┬──────────────┬──────────────────────────┤
│  Web App     │  Desktop     │   Mobile     │        CLI               │
│  (React)     │  (Electron)  │ (React Native)│      (Node.js)          │
└──────────────┴──────────────┴──────────────┴──────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│                     API GATEWAY (GraphQL Federation)                 │
│                  Authentication | Rate Limiting | Caching            │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                ┌─────────────────┼─────────────────┐
                ▼                 ▼                 ▼
┌────────────────┐   ┌────────────────┐   ┌────────────────┐
│  Core Services│   │   AI Services  │   │   Data Layer   │
├────────────────┤   ├────────────────┤   ├────────────────┤
│• User Service  │   │• LLM Gateway   │   │• PostgreSQL    │
│• Auth Service  │   │• Agent Runtime│   │• Redis Cache   │
│• Project Svc   │   │• Vector DB     │   │• Neo4j Graph   │
│• Task Service  │   │• Prompt Store  │   │• Chroma Vector│
│• Domain Svc    │   │• Model Market  │   │• IPFS Storage  │
│• Payment Svc   │   │• Training Svc  │   │• MinIO/S3      │
└────────────────┘   └────────────────┘   └────────────────┘
        │                    │                    │
        └────────────────────┼────────────────────┘
                             ▼
┌──────────────────────────────────────────────────────────────────────┐
│                   MESSAGE BUS (RabbitMQ/Kafka)                       │
│              Event-Driven Architecture for Microservices             │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│                     EXTERNAL INTEGRATIONS                            │
├──────────────┬──────────────┬──────────────┬──────────────────────────┤
│ • Stripe     │ • GitHub     │ • Slack      │ • Discord                │
│ • Twitter    │ • LinkedIn   │ • YouTube    │ • Google Drive           │
└──────────────┴──────────────┴──────────────┴──────────────────────────┘
```

Microservices List (45+ Services)

Category	Services
Core	API Gateway, Auth, User, Project, Task, Domain, Payment, Subscription
AI/ML	LLM Gateway, Agent Runtime, Vector DB, Embeddings, Training Pipeline, Model Registry
Storage	IPFS Pinner, S3 Manager, Database Proxy, Backup Service
Collaboration	Real-time Sync (Yjs), WebRTC, Comment Service, Notification
Marketplace	Listing Service, Escrow, Review System, Analytics
Web3	Wallet Service, Token Manager, DAO Governance, NFT Minting
Content	Blog Engine, SEO Manager, Social Publisher, Email Campaign
Dev Tools	Code Executor, Deploy Service, Domain Manager, SSL Cert
Analytics	Event Tracker, Metrics Aggregator, Report Generator
Security	Encryption Service, Audit Logger, Rate Limiter, WAF

---

4. Core Modules (A-Z)

Module A: Authentication & Identity

Purpose: Secure, flexible user authentication supporting traditional and Web3 methods

Features

- Email/Password with 2FA (TOTP, SMS, WebAuthn)
- OAuth providers (Google, GitHub, Twitter, LinkedIn)
- Web3 wallets (MetaMask, WalletConnect, Phantom)
- Passkey support (biometric login)
- Session management with device tracking
- Account recovery via social recovery (Web3)
- KYC/AML for marketplace transactions

Technical Specs

```
interface User {
  id: UUID
  email: string
  username: string
  walletAddresses: string[]
  profile: {
    name: string
    avatar: string
    bio: string
    website: string
    socialLinks: SocialLinks
  }
  settings: UserSettings
  roles: Role[]
  createdAt: Date
  lastActive: Date
  reputation: number
  tokenBalance: number
}
```

Module B: Domain & Idea Validator

Purpose: Validate business ideas and check domain availability with AI insights

Features

- Domain availability checker (500+ TLDs)
- AI-powered idea validation (market size, competition, viability)
- Domain suggestions with SEO scores
- Domain auction integration (Sedo, GoDaddy)
- Historical domain data (previous ownership, spam score)
- Trademark conflict detection
- Brand name generator with availability check

AI Prompt Template

```
prompt = f"""
Analyze the business idea: {idea}
Provide:
1. Viability Score (1-100)
2. Market Gap Analysis
3. Competitor Landscape
4. Recommended Domain Names
5. Potential Risks
6. Monetization Potential
"""
```

Module C: Smart Project Management (Custom Jira)

Purpose: AI-enhanced project management with visual planning

Features

- **Task Management:** Issues, epics, sprints, boards (Kanban/Scrum)
- **AI Workforce Planning:** Input team size → AI suggests task distribution
- **Resource Allocation:** Visual workload heatmaps
- **Time Tracking:** Manual and automatic tracking
- **Dependencies:** Blockers, relationships, Gantt charts
- **Custom Workflows:** Drag-and-drop workflow builder
- **Templates:** 50+ project templates (SaaS, ecommerce, mobile app, etc.)
- **Integration:** GitHub, GitLab, Figma, Slack

Visualizer Engine Specs

```
// React Three Fiber + WebGL
const Visualizer = {
  components: [
    'Workflow3D',      // 3D Kanban board
    'ResourceHeatmap', // Color-coded workload visualization
    'TimelineScroll',  // Interactive Gantt chart
    'DependencyGraph'  // Force-directed graph
  ],
  interactions: [
    'DragToAssign',    // Drag tasks to team members
    'VoiceCommands',   // "Move task 123 to in progress"
    'GestureControls'  // Pinch, zoom, rotate
  ]
}
```

Module D: AI Autoblogger & Content Engine

Purpose: Generate, optimize, and distribute content across platforms

Features

- **AutoBlogger:** Daily blog posts on trending topics
- **SEO Optimization:** Meta tags, keywords, readability scores
- **Multi-Format:** Blog posts, tweets, LinkedIn articles, newsletters
- **Content Calendar:** Schedule posts across platforms
- **Image Generation:** DALL-E/Stable Diffusion integration
- **Translation:** 50+ languages via AI
- **Plagiarism Check:** Ensure originality
- **RSS Feeds:** Auto-import from industry sources

Content Pipeline

```
Input: Topic/Keyword
  ↓
Research (web scraping + vector search)
  ↓
Outline Generation (AI)
  ↓
Content Writing (GPT-4/Llama)
  ↓
SEO Optimization
  ↓
Image Generation
  ↓
Multi-Platform Distribution
  ↓
Analytics Tracking
```

Module E: Knowledge Graph & Second Brain

Purpose: Connect everything—projects, people, ideas, resources

Features

- **Node Types:** Projects, Users, Tasks, Documents, Domains, Technologies
- **Relationships:** Belongs to, Depends on, Inspired by, Collaborates with
- **Visual Graph:** 3D interactive graph (React Three Fiber)
- **Auto-Connections:** AI suggests links between disconnected nodes
- **Bidirectional Links:** Wiki-style linking
- **Semantic Search:** Find concepts, not just keywords
- **Memory Palace:** Visual mind maps with spatial navigation
- **Export:** GraphML, JSON, PNG, SVG

Neo4j Schema

```
// Node Types
(:User {id, name, expertise})
(:Project {id, name, status})
(:Task {id, title, priority})
(:Document {id, title, content})
(:Domain {name, price, authority})
(:Technology {name, category})

// Relationships
(:User)-[:OWNS]->(:Project)
(:Project)-[:CONTAINS]->(:Task)
(:Task)-[:DEPENDS_ON]->(:Task)
(:Project)-[:USES]->(:Technology)
(:User)-[:COLLABORATES_WITH]->(:User)
(:Document)-[:MENTIONS]->(:Project)
```

Module F: AI Agent Runtime

Purpose: Run, manage, and market custom AI agents

Agent Types

Agent	Function	Tech Stack
Researcher	Web scraping, competitor analysis	LangChain, SerpAPI
Social Media	Content creation, scheduling	Tweepy, LinkedIn API
Code Buddy	Code review, debugging	Codex, LSP
Negotiator	Email outreach, follow-ups	Email API, NLP
SEO Optimizer	Keyword research, backlink analysis	Ahrefs API, SEMrush
Data Analyst	Report generation, insights	Pandas, Matplotlib
Customer Support	Ticket handling, FAQs	Rasa, Dialogflow

Agent Configuration

```yaml
agent:
  name: "ResearchBot"
  type: "researcher"
  model: "gpt-4"
  tools:
    - web_search
    - pdf_reader
    - data_analyzer
  schedule: "daily 9am"
  triggers:
    - new_project_created
    - competitor_detected
  permissions:
    - read:projects
    - write:reports
```

Module G: Marketplace & Digital Bazaar

Purpose: Buy, sell, and trade digital assets

Categories

1. **Prompts:** High-converting prompts for AI tools
2. **Templates:** Website, email, social media templates
3. **Code:** Snippets, micro-SaaS, plugins
4. **Domains:** Buy/sell domains with escrow
5. **Services:** Freelancer offerings (design, dev, marketing)
6. **Assets:** Logos, stock photos, videos, music
7. **Courses:** Tutorials, ebooks, guides

Escrow System

```
interface EscrowTransaction {
  id: UUID
  buyer: User
  seller: User
  amount: number
  currency: 'USD' | 'USDC' | 'NEXUS'
  status: 'pending' | 'funded' | 'released' | 'disputed'
  deadline: Date
  arbitrator?: User
  releaseConditions: Condition[]
}
```

Module H: Decentralized Storage (IPFS)

Purpose: Immutable, distributed storage for user data

Features

- **Auto-Pinning:** All user content automatically pinned to IPFS
- **Self-Hostable:** Users can run IPFS nodes
- **Encrypted Sharing:** End-to-end encrypted file sharing
- **Versioning:** Full history of all changes
- **Bandwidth Optimization:** Local caching + CDN
- **File Types:** Documents, images, videos, archives

IPFS Integration

```
// File upload flow
const file = await uploadToIPFS(buffer)
const hash = file.cid.toString()
await saveToDatabase({
  cid: hash,
  filename: name,
  encryptionKey: userPublicKey
})
return `ipfs://${hash}`
```

Module I: Low-Code/No-Code Builder (The Forge)

Purpose: Build and deploy applications without code

Components

- **Drag-Drop Editor:** 100+ pre-built components
- **AI Builder:** "Build me a landing page with payment"
- **Database Builder:** Visual schema designer
- **API Connector:** Connect to any REST/GraphQL API
- **Workflow Builder:** Visual automation (Zapier alternative)
- **Form Builder:** Drag-drop forms with validation
- **Deployment:** One-click deploy to subdomain

Generated Stack

- Frontend: React/Next.js
- Backend: Node.js/Express
- Database: PostgreSQL
- Hosting: Auto-deployed to Vercel-like infrastructure

Module J: Collaboration Suite

Purpose: Real-time collaborative tools

Tools

1. **Collaborative Docs:** Yjs-based real-time editor
2. **Whiteboard:** Infinite canvas (Excalidraw-based)
3. **Code Playground:** VS Code in browser with collaboration
4. **Video Calls:** WebRTC (Jitsi) with screen sharing
5. **Comments:** Threaded discussions on any entity
6. **Approvals:** Multi-signature approvals for workflows

Real-time Architecture

```
// WebSocket + Yjs CRDT
import * as Y from 'yjs'
import { WebsocketProvider } from 'y-websocket'

const doc = new Y.Doc()
const provider = new WebsocketProvider(
  'ws://localhost:1234',
  'room-id',
  doc
)

// Shared text
const text = doc.getText('document')
text.observe(() => {
  // Sync to UI
})
```

Module K: Web3 & DAO Toolkit

Purpose: Launch and manage decentralized organizations

Features

- **DAO Creator:** Deploy DAO contracts (Aragon, Moloch)
- **Token Launch:** Create ERC20 tokens
- **NFT Minting:** Create collections (ERC721/ERC1155)
- **Voting Systems:** Snapshot, on-chain voting
- **Treasury:** Multi-sig wallet management
- **Governance Analytics:** Voter participation, proposals
- **Legal Wrapper:** Entity formation (LLC, foundation)

Smart Contract Templates

```
// DAO Template
contract NexusDAO {
    mapping(address => uint256) public balances;
    mapping(uint256 => Proposal) public proposals;

    struct Proposal {
        string description;
        uint256 votesFor;
        uint256 votesAgainst;
        bool executed;
    }

    function createProposal(string memory description) external {
        // Proposal creation logic
    }

    function vote(uint256 proposalId, bool support) external {
        // Voting logic
    }
}
```

Module L: Gamification & Learning

Purpose: Make skill development engaging and rewarding

Features

- **Skill Trees:** Visual skill progression (RPG-style)
- **Quests:** Time-limited challenges with rewards
- **Achievements:** Badges for milestones
- **Leaderboards:** Community rankings
- **Certifications:** Verified credentials
- **Mentorship:** Connect learners with experts
- **XP System:** Experience points for actions

Gamification Mechanics

```
const actions = {
  create_project: 100,
  complete_task: 50,
  publish_blog: 200,
  help_community: 150,
  refer_friend: 500,
  get_star_on_github: 1000
}
```

Module M: Encrypted Vault

Purpose: Secure storage for sensitive business data

Features

- **End-to-End Encryption:** Signal Protocol (Double Ratchet)
- **File Types:** Any file type supported
- **Zero-Knowledge:** Server never sees decrypted data
- **Emergency Access:** Time-locked recovery
- **Audit Logs:** All access attempts logged
- **Hardware Wallet:** Ledger/Trezor support

Encryption Flow

```
User Keypair Generation
  ↓
Public Key → Server (for sharing)
Private Key → Stored locally (encrypted with password)
  ↓
Data Encryption (AES-256-GCM)
  ↓
Key Exchange (X25519)
  ↓
Ratchet for forward secrecy
```

Module N: Multi-Platform Clients

Purpose: Access from anywhere

Platforms

Platform	Tech Stack	Features
Web	React, Next.js	PWA, offline support
Desktop	Electron, Tauri	System tray, native notifications
Mobile	React Native	Biometric auth, camera, location
CLI	Node.js, Commander	Scripting, automation, CI/CD

Offline-First Strategy

```
// Offline sync with IndexedDB
const offlineDB = new Dexie('NexusDB')
offlineDB.version(1).stores({
  projects: 'id, name, updatedAt',
  tasks: 'id, projectId, title'
})

// Sync when online
navigator.serviceWorker.addEventListener('online', async () => {
  const pending = await offlineDB.pending.toArray()
  await syncToServer(pending)
})
```

Module O: Marketing Automation

Purpose: Automate social media and content distribution

Features

- **Social Media Bots:** LinkedIn, Twitter, Instagram, Facebook
- **Content Scheduler:** Calendar view
- **Analytics:** Engagement tracking
- **A/B Testing:** Test headlines, images
- **Email Marketing:** Newsletter automation
- **SEO Tools:** Rank tracking, backlink monitoring

Bot Configuration

```
class SocialMediaBot:
    def __init__(self, platforms):
        self.platforms = platforms

    def post_content(self, content):
        for platform in self.platforms:
            try:
                platform.post(content)
                self.log_success(platform)
            except Exception as e:
                self.log_error(platform, e)

    def schedule(self, schedule):
        # Cron-based scheduling
        pass
```

Module P: Notification & Reminder Engine

Purpose: Keep users engaged with timely notifications

Channels

- **Push:** OneSignal, Web Push, Mobile Push
- **Email:** SendGrid, AWS SES
- **SMS:** Twilio
- **In-App:** Toast notifications
- **Webhook:** Custom integrations

Smart Scheduling

```
class SmartReminder {
  async schedule(task: Task) {
    // AI predicts optimal reminder time
    const optimalTime = await predictEngagementTime(user)

    await createReminder({
      user: user.id,
      message: `Task "${task.title}" is due soon`,
      time: optimalTime,
      channels: ['push', 'email']
    })
  }

  async trackEngagement() {
    // Adjust based on open rates
  }
}
```

Module Q: Analytics & Business Intelligence

Purpose: Track everything, measure everything

Metrics Tracked

- User engagement (DAU, MAU, session duration)
- Feature adoption
- Conversion funnels
- Revenue analytics
- Content performance
- System performance

Custom Dashboards

```
// Drag-drop dashboard builder
const Dashboard = {
  widgets: [
    'UserGrowthChart',
    'RevenueTable',
    'TopProjectsList',
    'AIUsageHeatmap'
  ],
  dataSources: [
    'PostgreSQL',
    'ClickHouse',
    'Mixpanel',
    'CustomEvents'
  ]
}
```

Module R: Sustainability & Carbon Tracking

Purpose: Build eco-friendly digital products

Features

- **Carbon Calculator:** Estimate digital carbon footprint
- **Green Recommendations:** Optimize for energy efficiency
- **Offset Marketplace:** Purchase verified carbon offsets
- **Eco-Badge:** Verified sustainable projects
- **Green Hosting:** Recommend eco-friendly providers

Calculation Model

```
interface CarbonFootprint {
  serverEmissions: number  // kg CO2
  networkEmissions: number
  userDeviceEmissions: number
  total: number

  recommendations: {
    optimizeImages: boolean
    useCDN: boolean
    greenHosting: string
  }
}
```

Module S: DevOps & Deployment

Purpose: Deploy and manage applications

Features

- **Free Subdomains:** *.nexus.app for all projects
- **Custom Domains:** Bring your own domain
- **SSL Certificates:** Auto-provision Let's Encrypt
- **CI/CD Pipeline:** GitHub/GitLab integration
- **Environment Variables:** Manage secrets
- **Log Management:** Centralized logging
- **Metrics:** CPU, memory, request rates

Deployment Flow

```
# .nexus/deploy.yml
name: My App
type: static  # or 'node', 'python', 'docker'
build:
  command: npm run build
  output: dist
env:
  NODE_ENV: production
domains:
  - myapp.nexus.app
  - custom.com
```

Module T: Community & Support

Purpose: Build a thriving community

Features

- **Forum:** Discourse-like with AI moderation
- **Knowledge Base:** Searchable documentation
- **Support Tickets:** Integrated helpdesk
- **Live Chat:** Support with AI pre-screening
- **User Groups:** Special interest groups
- **Events:** Hackathons, webinars, meetups

Moderation System

```
class ContentModerator {
  async moderate(content: string) {
    // AI detection
    const toxicity = await detectToxicity(content)
    const spam = await detectSpam(content)

    if (toxicity > 0.8 || spam > 0.9) {
      await flagForReview(content)
      return 'flagged'
    }

    return 'approved'
  }
}
```

---

5. Technical Stack

Frontend Stack

Layer	Technology	Purpose
Framework	React 18 + Next.js 14	SSR, routing, performance
State	Zustand + TanStack Query	Global state, server state
UI	TailwindCSS + Radix UI	Styling, components
3D	React Three Fiber	Visualizations, graphs
Real-time	Socket.io + Yjs	Collaboration
PWA	Workbox	Offline support

Backend Stack

Layer	Technology	Purpose
API	GraphQL (Apollo Federation)	Unified gateway
Services	Node.js + TypeScript	Microservices
Queue	BullMQ + Redis	Background jobs
Auth	Passport.js + JWT	Authentication
Payments	Stripe + Coinbase Commerce	Monetization

AI/ML Stack

Component	Technology
LLM	GPT-4, Claude, Llama 3
Framework	LangChain, LlamaIndex
Vector DB	Chroma, Pinecone (optional)
Embeddings	OpenAI, Cohere
Training	PyTorch, Hugging Face
Orchestration	Flowise, LangFlow

Database Stack

Database	Purpose
PostgreSQL	Primary data, transactions
Redis	Cache, sessions, queues
Neo4j	Knowledge graph
Chroma	Vector embeddings
ClickHouse	Analytics, time-series
MinIO	Object storage (S3 compatible)

DevOps Stack

Tool	Purpose
Docker	Containerization
Kubernetes	Orchestration
GitHub Actions	CI/CD
Terraform	Infrastructure as Code
Prometheus	Monitoring
Grafana	Visualization
ELK Stack	Logging

---

6. Database Schema

PostgreSQL Schema

```sql
-- Users
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255),
    wallet_addresses TEXT[],
    profile JSONB,
    reputation INT DEFAULT 0,
    token_balance DECIMAL DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW(),
    last_active TIMESTAMP
);

-- Projects
CREATE TABLE projects (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    owner_id UUID REFERENCES users(id),
    name VARCHAR(100) NOT NULL,
    description TEXT,
    status VARCHAR(20) DEFAULT 'active',
    settings JSONB,
    metadata JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Tasks (Jira-like)
CREATE TABLE tasks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID REFERENCES projects(id),
    assignee_id UUID REFERENCES users(id),
    title VARCHAR(200) NOT NULL,
    description TEXT,
    status VARCHAR(20) DEFAULT 'todo',
    priority VARCHAR(10),
    story_points INT,
    due_date DATE,
    parent_id UUID REFERENCES tasks(id),
    created_at TIMESTAMP DEFAULT NOW()
);

-- Domain Searches
```

```sql
CREATE TABLE domain_searches (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    domain_name VARCHAR(255),
    available BOOLEAN,
    price DECIMAL,
    ai_analysis JSONB,
    searched_at TIMESTAMP DEFAULT NOW()
);

-- Marketplace Listings
CREATE TABLE listings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    seller_id UUID REFERENCES users(id),
    title VARCHAR(200),
    description TEXT,
    category VARCHAR(50),
    price DECIMAL,
    currency VARCHAR(10),
    files JSONB,
    status VARCHAR(20),
    created_at TIMESTAMP DEFAULT NOW()
);

-- AI Agents
CREATE TABLE agents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    owner_id UUID REFERENCES users(id),
    name VARCHAR(100),
    type VARCHAR(50),
    configuration JSONB,
    schedule JSONB,
    last_run TIMESTAMP,
    status VARCHAR(20)
);

-- Notifications
CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    type VARCHAR(50),
    title VARCHAR(200),
    content TEXT,
```

```
    metadata JSONB,
    read BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Analytics Events
CREATE TABLE events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    event_type VARCHAR(100),
    properties JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);
```

Neo4j Graph Schema

```
// Indexes
CREATE INDEX user_id FOR (u:User) ON (u.id)
CREATE INDEX project_id FOR (p:Project) ON (p.id)

// Constraints
CREATE CONSTRAINT unique_user_id FOR (u:User) REQUIRE u.id IS UNIQUE
CREATE CONSTRAINT unique_project_id FOR (p:Project) REQUIRE p.id IS UNIQUE

// Relationships
CREATE (u:User {id: 'user1'})-[:OWNS]->(p:Project {id: 'proj1'})
CREATE (p)-[:CONTAINS]->(t:Task {id: 'task1'})
CREATE (t)-[:DEPENDS_ON]->(t2:Task)
CREATE (u)-[:COLLABORATES_WITH]->(u2:User)
CREATE (p)-[:USES]->(tech:Technology {name: 'React'})
```

---

7. API Specifications

GraphQL Schema (Federated)

```
type User @key(fields: "id") {
  id: ID!
  email: String!
  username: String!
  profile: Profile
  projects: [Project!]
  tokenBalance: Float!
  reputation: Int!
}

type Project {
  id: ID!
  name: String!
  description: String
  owner: User!
  tasks: [Task!]
  status: ProjectStatus!
  createdAt: DateTime!
  knowledgeGraph: GraphNode!
}

type Task {
  id: ID!
  title: String!
  description: String
  assignee: User
  status: TaskStatus!
  priority: Priority!
  storyPoints: Int
  dependencies: [Task!]
  comments: [Comment!]
}

type DomainCheckResult {
  domain: String!
  available: Boolean!
  price: Float
  suggestions: [String!]
  aiAnalysis: AIAnalysis!
}

type AIAnalysis {
```

```
  viabilityScore: Int!
  marketGap: String!
  competitors: [String!]
  recommendations: [String!]
}

type MarketplaceListing {
  id: ID!
  title: String!
  description: String!
  seller: User!
  price: Float!
  category: ListingCategory!
  files: [File!]
  reviews: [Review!]
}

type Query {
  me: User!
  project(id: ID!): Project
  projects(limit: Int, offset: Int): [Project!]
  domainCheck(domain: String!): DomainCheckResult!
  searchDomains(query: String!): [DomainCheckResult!]
  marketplace(category: ListingCategory): [MarketplaceListing!]
  agents: [Agent!]
  analytics(days: Int): AnalyticsData!
}

type Mutation {
  createProject(input: CreateProjectInput!): Project!
  updateTask(input: UpdateTaskInput!): Task!
  generateBlog(topic: String!): BlogPost!
  createListing(input: CreateListingInput!): MarketplaceListing!
  purchaseListing(id: ID!): Transaction!
  createAgent(input: CreateAgentInput!): Agent!
  runAgent(id: ID!): AgentRunResult!
}

type Subscription {
  taskUpdated(projectId: ID!): Task!
  notification: Notification!
```

```
  collaboration(roomId: ID!): CollaborationEvent!
}
```

REST Endpoints (for webhooks, files)

```
# File Upload
POST /api/upload
Content-Type: multipart/form-data

# Webhook Receiver
POST /api/webhooks/github
POST /api/webhooks/stripe
POST /api/webhooks/gitlab

# Export Data
GET /api/export/project/:id
GET /api/export/analytics/:period
```

---

8. Frontend Architecture

Component Structure

```
src/
├── app/                    # Next.js app router
│   ├── (auth)/            # Auth routes
│   ├── (dashboard)/       # Main app
│   ├── marketplace/       # Marketplace
│   ├── community/         # Forum
│   └── api/               # API routes
├── components/
│   ├── ui/                # Reusable UI (shadcn)
│   ├── dashboard/         # Dashboard components
│   ├── projects/          # Project management
│   ├── ai/                # AI interfaces
│   ├── marketplace/       # Marketplace UI
│   ├── visualizer/        # 3D visualizations
│   └── collaboration/     # Real-time features
├── lib/
│   ├── graphql/           # GraphQL queries
│   ├── store/             # Zustand stores
│   ├── api/               # API clients
│   └── utils/             # Helpers
├── hooks/                 # Custom React hooks
├── styles/                # Global styles
└── types/                 # TypeScript types
```

State Management

```
// Zustand store example
interface AppStore {
  user: User | null
  projects: Project[]
  currentProject: Project | null
  notifications: Notification[]

  setUser: (user: User) => void
  addProject: (project: Project) => void
  updateTask: (task: Task) => void
  addNotification: (notification: Notification) => void
}

export const useAppStore = create<AppStore>((set) => ({
  user: null,
  projects: [],
  currentProject: null,
  notifications: [],

  setUser: (user) => set({ user }),
  addProject: (project) => set((state) => ({
    projects: [...state.projects, project]
  })),
  updateTask: (task) => set((state) => ({
    projects: state.projects.map(p =>
      p.id === task.projectId
        ? { ...p, tasks: p.tasks.map(t => t.id === task.id ? task : t) }
        : p
    )
  })),
  addNotification: (notification) => set((state) => ({
    notifications: [notification, ...state.notifications]
  }))
}))
```

---

9. Security Implementation

Encryption Standards

Use Case	Algorithm	Key Size
Data at rest	AES-256-GCM	256-bit
Data in transit	TLS 1.3	256-bit
User passwords	bcrypt	12 rounds
Vault encryption	X25519 + AES-256	256-bit
JWT signing	RS256	2048-bit
Post-quantum	CRYSTALS-Kyber	1024-bit

Security Features

```
// Rate limiting
import rateLimit from 'express-rate-limit'

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limit each IP to 100 requests
  message: 'Too many requests'
})

// Input validation
import { z } from 'zod'

const projectSchema = z.object({
  name: z.string().min(1).max(100),
  description: z.string().max(1000).optional(),
  status: z.enum(['active', 'archived', 'deleted'])
})

// CORS
const corsOptions = {
  origin: process.env.ALLOWED_ORIGINS?.split(','),
  credentials: true,
  optionsSuccessStatus: 200
}

// Helmet for security headers
import helmet from 'helmet'
app.use(helmet())
```

Audit Logging

```
CREATE TABLE audit_logs (
    id UUID PRIMARY KEY,
    user_id UUID REFERENCES users(id),
    action VARCHAR(100),
    resource_type VARCHAR(50),
    resource_id UUID,
    old_values JSONB,
    new_values JSONB,
    ip_address INET,
    user_agent TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);
```

---

10. Open Source Strategy

Licensing Structure

Component	License	Reason
Core Platform	AGPL-3.0	Ensure modifications remain open
UI Components	MIT	Maximum adoption
SDK/Libraries	Apache 2.0	Commercial friendly
Documentation	CC BY-SA 4.0	Share improvements
Templates	MIT	Encourage reuse

Contribution Model

```
# .github/CONTRIBUTING.md
levels:
  - first-time: Good first issues
  - regular: Feature development
  - core: Architecture decisions
  - maintainer: Merge rights

rewards:
  - tokens: Per PR merged
  - swag: 50+ PRs
  - travel: 500+ PRs (to conferences)
  - revenue: Top 10 contributors get 0.5% each
```

Governance

```
// DAO voting power
interface VotingPower {
  tokens: number // 1 token = 1 vote
  contributions: number // 100 contributions = 1 vote
  timeHeld: number // Locked tokens get multiplier
}

// Proposal types
enum ProposalType {
  FEATURE_REQUEST,
  BUDGET_ALLOCATION,
  MAINTAINER_ELECTION,
  LICENSE_CHANGE
}
```

---

11. Development Roadmap

Phase 1: Foundation (Months 1-3)

Goal: Core infrastructure + MVP features

Week	Milestone	Deliverables
1-2	Setup	GitHub repo, CI/CD, Docker setup, Discord
3-4	Auth	User auth, OAuth, Web3 wallets
5-6	Projects	Create/read/update projects, basic tasks
7-8	Domain Validator	Domain checking API, AI analysis
9-10	Database	PostgreSQL, Redis, basic schemas
11-12	MVP Launch	Beta users, initial feedback

Phase 2: Core Features (Months 4-6)

Goal: AI features + marketplace + community

Week	Milestone	Deliverables
13-14	AI Integration	LangChain setup, GPT-4, vector DB
15-16	Autoblogger	Content generation, SEO tools
17-18	Marketplace	Listings, escrow, payments
19-20	Community	Forum, real-time chat, moderation
21-22	Knowledge Graph	Neo4j, visual graph, semantic search
23-24	Public Launch	Product Hunt, marketing push

Phase 3: Advanced Features (Months 7-9)

Goal: Collaboration + Web3 + gamification

Week	Milestone	Deliverables
25-26	Collaboration	Real-time docs, whiteboard, video
27-28	Web3	DAO toolkit, NFTs, token launch
29-30	Gamification	XP, achievements, leaderboards
31-32	Low-Code Builder	Drag-drop builder, templates
33-34	Mobile Apps	iOS/Android apps (React Native)
35-36	Desktop Apps	Electron, Tauri builds

Phase 4: Scale & Monetize (Months 10-12)

Goal: Enterprise features + sustainability

Week	Milestone	Deliverables
37-38	Enterprise	SSO, audit logs, compliance
39-40	Analytics	BI dashboards, reporting
41-42	DevOps	Kubernetes, auto-scaling
43-44	Monetization	Subscription tiers, usage billing
45-46	Security	Audit, bug bounty, pentest
47-48	1.0 Launch	Stable release, case studies

---

12. Deployment Infrastructure

Kubernetes Architecture

```yaml
# deployment/k8s/namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: nexus-os

---
# deployment/k8s/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nexus-ingress
  namespace: nexus-os
spec:
  tls:
  - hosts:
    - api.nexus.app
    secretName: nexus-tls
  rules:
  - host: api.nexus.app
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: api-gateway
            port:
              number: 80
```

Resource Requirements

Service	CPU	Memory	Replicas
API Gateway	1 core	2GB	3
Auth Service	0.5 core	1GB	2
AI Orchestrator	4 cores	8GB	2
Vector DB	2 cores	4GB	1
PostgreSQL	2 cores	4GB	1 (primary)
Redis	1 core	2GB	1 (cluster)
Neo4j	2 cores	8GB	1
Worker Queue	1 core	2GB	3-10 (auto)

Cost Estimation

Service	Monthly Cost (Est.)
Cloud (AWS/GCP)	$2,000-5,000$
AI API (GPT-4)	$1,000-10,000$
CDN	$500
Object Storage	$500
Monitoring	$300
Total	$4,300-**16,300**

---

13. Testing Strategy

Testing Pyramid

```
        /\
       /E2E\         5% - Critical user flows
      /------\
     /Integration\   20% - API, DB, service integration
    /------------\
   /   Unit Tests   \ 75% - Components, functions, utils
  /------------------\
```

Test Coverage Requirements

```
// Jest configuration
module.exports = {
  coverageThreshold: {
    global: {
      statements: 80,
      branches: 75,
      functions: 85,
      lines: 80
    }
  }
}
```

Test Types

Type	Tools	Target
Unit	Jest, Vitest	All functions
Integration	Supertest, TestContainers	API endpoints
E2E	Playwright, Cypress	Critical paths
Load	K6, Artillery	1000+ concurrent
Security	OWASP ZAP, Snyk	Monthly scans

---

14. Monitoring & Observability

Metrics Dashboard

```
# Prometheus metrics to track
metrics:
  - http_requests_total
  - http_request_duration_seconds
  - database_connection_pool
  - queue_length
  - ai_api_latency
  - user_active_sessions
  - error_rate
  - memory_usage
  - cpu_usage
```

Grafana Dashboards

```
const dashboards = [
  'System Health (CPU, Memory, Network)',
  'API Performance (Latency, Error Rate)',
  'Business Metrics (Users, Projects, Revenue)',
  'AI Usage (Tokens, Cost, Latency)',
  'Database Performance (Queries, Connections)'
]
```

Alerting Rules

```
alerts:
  - name: HighErrorRate
    condition: error_rate > 5% for 5m
    severity: critical
    channels: [pagerduty, slack]

  - name: SlowAPI
    condition: p95_latency > 2s for 5m
    severity: warning
    channels: [slack]

  - name: HighAICost
    condition: ai_cost > $100/hour
    severity: warning
    channels: [email, slack]
```

15. Monetization Model

Revenue Streams

Stream	Model	Projected (Year 1)
Cloud Hosting	Freemium (Free tier + $29/mo Pro)	$100,000
Marketplace	5% transaction fee	$50,000
Enterprise	$500/mo (SLA, SSO, compliance)	$200,000
Support Contracts	$5,000/mo for dedicated support	$150,000
White-label	$2,000/mo for rebranding	$100,000
Token Sale	Utility tokens (NEXUS)	$500,000
Total		**$1,100,000**

Pricing Tiers

Tier	Price	Features
Free	$0	1 project, basic AI, community support
Pro	$29/mo	Unlimited projects, advanced AI, marketplace access
Business	$99/mo	Team features, analytics, priority support
Enterprise	$499/mo	White-label, SLA, compliance, dedicated support

---

16. Risk Management

Risk	Probability	Impact	Mitigation
Technical Debt	High	High	Weekly refactoring, code reviews, documentation
Security Breach	Low	Critical	Regular audits, bug bounty, encryption
AI Cost Explosion	Medium	High	Rate limiting, caching, optimization
Competitor	Medium	High	Open source differentiation, community lock-in
Adoption Slow	Medium	Medium	Marketing push, case studies, referrals
Contributor Burnout	Medium	Medium	Recognition, compensation, rotation
Scalability	Medium	High	Kubernetes, auto-scaling, performance testing

---

17. Success Metrics

Key Performance Indicators (KPIs)

Category	Metric	Target (Year 1)
User Growth	Total Users	100,000
	MAU	25,000
	DAU	5,000
Engagement	Session Duration	30+ minutes
	Features per User	5+
Community	GitHub Stars	10,000
	Contributors	500
	Discord Members	50,000
Business	MRR	$50,000
	Marketplace GMV	$1,000,000
Technical	Uptime	99.9%
	API Latency (p95)	<200ms
	Deploy Time	<10 minutes

18. Community Guidelines

Code of Conduct

```
# Contributor Covenant Code of Conduct

## Our Pledge
We as members, contributors, and leaders pledge to make participation in our
community a harassment-free experience for everyone, regardless of age, body
size, visible or invisible disability, ethnicity, sex characteristics, gender
identity and expression, level of experience, education, socio-economic status,
nationality, personal appearance, race, religion, or sexual identity
and orientation.

## Our Standards
Examples of behavior that contributes to a positive environment:
* Using welcoming and inclusive language
* Being respectful of differing viewpoints
* Gracefully accepting constructive criticism
* Focusing on what is best for the community
```

Contribution Workflow

1. **Fork** the repository
2. **Create branch** (`git checkout -b feature/amazing-feature`)
3. **Commit changes** (`git commit -m 'Add amazing feature'`)
4. **Push** (`git push origin feature/amazing-feature`)
5. **Open Pull Request** with template filled
6. **Code Review** (2 approvals required)
7. **CI passes** (tests, lint, build)
8. **Merge** (squash and merge)

PR Review Guidelines

```
interface PRChecklist {
  tests: boolean
  documentation: boolean
  noBreakingChanges: boolean
  followsStyleGuide: boolean
  passesCI: boolean
  hasScreenshots: boolean (for UI changes)
}
```

---

19. Documentation Requirements

Documentation Structure

```
docs/
├── README.md
├── CONTRIBUTING.md
├── CODE_OF_CONDUCT.md
├── LICENSE
├── getting-started/
│   ├── installation.md
│   ├── quick-start.md
│   └── configuration.md
├── user-guide/
│   ├── projects.md
│   ├── tasks.md
│   ├── ai-features.md
│   └── marketplace.md
├── developer-guide/
│   ├── architecture.md
│   ├── api-reference.md
│   ├── database-schema.md
│   └── microservices.md
├── deployment/
│   ├── docker.md
│   ├── kubernetes.md
│   └── self-hosting.md
└── api/
    ├── graphql.md
    ├── rest.md
    └── webhooks.md
```

API Documentation

```
# Auto-generated GraphQL docs
"""
Get project details with tasks and collaborators
"""
type Query {
  project(id: ID!): Project
    @auth(requires: [USER])
    @rateLimit(limit: 100, duration: 60)
}
```

---

20. Launch Checklist

Pre-Launch (Week -2)

- [ ] Security audit completed
- [ ] Load testing passed (1000 concurrent users)
- [ ] Documentation complete
- [ ] Terms of Service & Privacy Policy
- [ ] Stripe/Coinbase integration tested
- [ ] Backup & disaster recovery tested

Launch Day (Week 0)

- [ ] Product Hunt launch scheduled
- [ ] Hacker News post ready
- [ ] Discord/Twitter announcements
- [ ] Blog post published
- [ ] Email newsletter to beta users
- [ ] 24/7 on-call support ready

Post-Launch (Week +1-4)

- [ ] Monitor error rates
- [ ] Collect user feedback
- [ ] Fix critical bugs within 24h
- [ ] Weekly feature releases
- [ ] Community highlights
- [ ] Case studies with early users

🎯 Final Note

NEXUS OS is not just software—it's a movement to democratize digital creation. By combining open source principles with cutting-edge AI, Web3 technology, and a vibrant community, we're building something that will empower millions of creators, entrepreneurs, and businesses worldwide.

Get Involved

- **GitHub:** github.com/nexus-os/nexus
- **Discord:** discord.gg/nexus-os
- **Twitter:** @NexusOS
- **Website:** nexus.app

Built with ❤️ by the community, for the community.